Федеральное государственное бюджетное образовательное учреждение высшего образования
«Херсонский государственный педагогический университет»

Институт информационных технологий

Кафедра программной инженерии и информационных технологий

ПРАКТИЧЕСКАЯ РАБОТА
по дисциплине:
«Программирование графических процессоров»

на тему:
«Практическая работа 6. Типы памяти. Разделяемая память»

Выполнил:
Студент гр.: 12-25РПм
Трифонов Д.С.

Проверил
Преподаватель:
Алешов Е.В.

Оценка:_____
Дата: 10.11.2026

Херсон, 2026

ПРАКТИЧЕСКАЯ РАБОТА

1. Цель работы

Изучить типы памяти GPU в CUDA и особенности разделяемой памяти (__shared__), синхронизацию потоков (__syncthreads__), ограничения размера shared memory на блок. Реализовать программы скалярного произведения, численного интегрирования и вычисления евклидовой нормы вектора с использованием разделяемой памяти.

2. Условия выполнения

Работа выполнялась на ноутбуке DELL с интегрированной графикой Intel HD Graphics 3000 (без дискретного NVIDIA GPU). Полноценный запуск CUDA-ядер на устройстве невозможен, поэтому алгоритмы реализованы минималистично: CUDA-исходник app/pr6_shared_memory.cu и CPU-эмуляция ядер с разделяемой памятью (app/main.py, app/cuda_emu.py). Объёмы данных вымышленные и небольшие: N = 512, BLOCK_SIZE = 128 (число нитей равно числу элементов shared-массива; 2×128×8 = 2048 байт на блок при лимите 48 КБ).

3. Ход и результаты выполнения

3.1. Задание 1. Скалярное произведение двух векторов

Реализовано ядро scalMult по методичке: два массива в __shared__, копирование из глобальной памяти, __syncthreads(), редукция произведений одной нитью блока. Векторы A и B длины 512 сформированы вымышленными данными (линейный тренд + шум). Время ядра (эмуляция) ≈ 0.62 мс. Результат совпал с проверкой numpy.dot (разность порядка 10^{-13}).

Фрагмент ядра (CUDA):

```
__global__ void scalMult(const BASE_TYPE *A, const BASE_TYPE *B,
                         BASE_TYPE *C, int numElem) {
    __shared__ BASE_TYPE ash[BLOCK_SIZE];
    __shared__ BASE_TYPE bsh[BLOCK_SIZE];
    ash[threadIdx.x] = A[blockIdx.x * blockDim.x + threadIdx.x];
    bsh[threadIdx.x] = B[blockIdx.x * blockDim.x + threadIdx.x];
    __syncthreads();
    if (threadIdx.x == 0) {
        BASE_TYPE sum = 0.0;
        for (int j = 0; j < blockDim.x; j++)
            sum += ash[j] * bsh[j];
        C[blockIdx.x] = sum;
    }
}
```



Рисунок 1 – Результат скалярного произведения (shared memory)

3.2. Задания 2–3. Определённый интеграл (прямоугольники, трапеции, Симпсон)

Вычислялся интеграл $f(x) = x^2 \cdot \cos(x)$ на отрезке $[0, \pi/2]$ при $N = 256$ интервалах. Значения подынтегральной функции (с весами квадратур) суммировались редукцией через разделяемую память. Точное значение: $\pi^2/4 - 2 \approx 0.4674011003$. Сравнение точности: центральные прямоугольники — ошибка $\approx 3.9 \cdot 10^{-6}$; трапеции — $\approx 7.7 \cdot 10^{-6}$; формула Симпсона — $\approx 2.8 \cdot 10^{-11}$ (наибольшая точность).

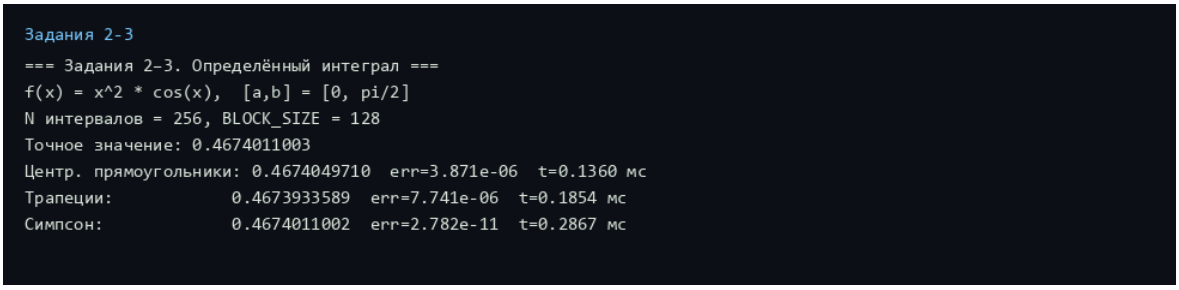


Рисунок 2 – Сравнение квадратурных формул

3.3. Задание 4. Евклидова норма вектора

Норма вычислялась как $\|v\| = \sqrt{\langle v, v \rangle}$: сначала скалярное произведение вектора на себя ядром с shared memory, затем квадратный корень. Для вымышленного вектора длины 512 получено $\|v\| \approx 66.8389391109$; расхождение с numpy.linalg.norm отсутствует.

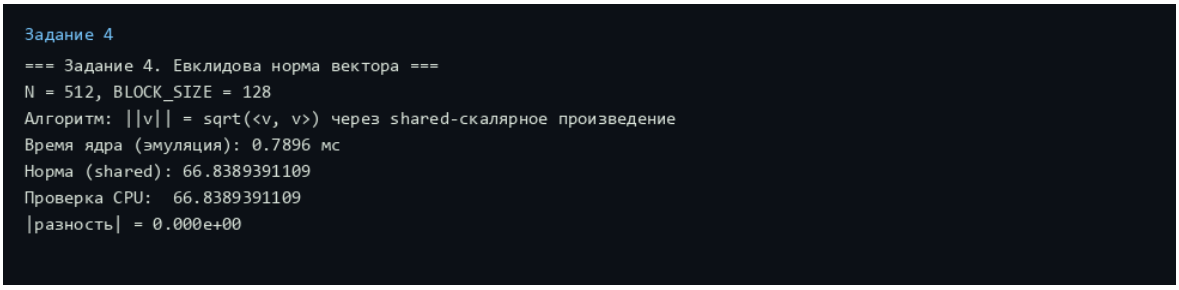


Рисунок 3 – Евклидова норма через скалярное произведение

3.4. Сводный прогон программы

Запуск: python app/main.py. Ниже — полный вывод минималистичного прогона на CPU-эмуляции.

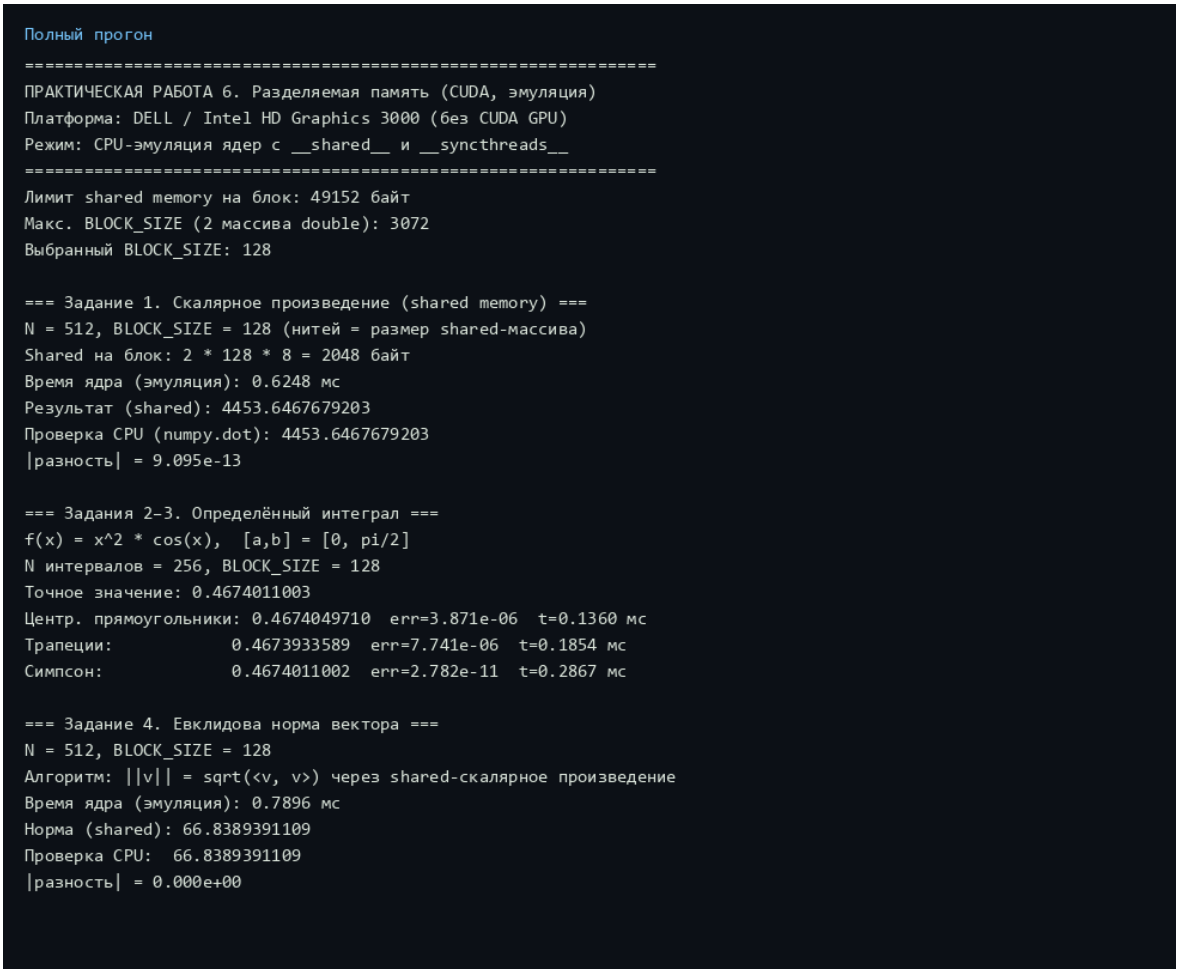


Рисунок 4 – Полный прогон программы

4. Ответы на контрольные вопросы

1. В технологии CUDA выделяют следующие типы памяти: регистровая, локальная, разделяемая (shared), глобальная, константная и текстурная. Они различаются областью видимости (нить / блок / решётка), скоростью доступа и правами чтение/запись.

2. Для задания разделяемой памяти применяется спецификатор __shared__. Основные особенности: память выделяется на блок и доступна всем нитям блока; скорость доступа высокая (близка к регистрам); оптимизирована под одновременное обращение половины нитей варпа; разбита на 32 банка по 32-битных слова; часто требует явной синхронизации __syncthreads(); размер ограничен (типично до 48 КБ на блок); при неудачном доступе возможны конфликты банков.

5. Вывод

В ходе работы изучены типы памяти CUDA и практика использования разделяемой памяти. Реализованы скалярное произведение, три квадратурные формулы интегрирования и евклидова норма вектора с учётом ограничения размера shared memory и равенства числа нитей размеру shared-массива. Наилучшую точность интегрирования показала формула Симпсона. Из-за отсутствия CUDA GPU на Intel HD 3000 расчёты выполнены в эмуляции при сохранении структуры ядер методички; исходный CUDA-код сохранён в app/pr6_shared_memory.cu.